

Neural Networks

Activations, Architecture, Feed-Forward, Backpropagation, and
Parameter Updates

A mathematical and worked-example guide

A neural network can be understood as a parameterized function built by repeatedly applying a linear transformation followed by a nonlinear activation. This guide starts at the level of one neuron and builds up to complete forward and backward passes, including matrix dimensions, gradients, optimization, mini-batches, initialization, and practical design choices.

Contents

1	The Core Idea: One Neuron	4
1.1	Numerical example	4
2	Why Activation Functions Are Necessary	4
3	Common Activation Functions	5
3.1	ReLU	5
3.2	Sigmoid	6
3.3	Tanh	6
3.4	GELU	7
4	Why Activations Usually Follow Every Hidden Layer	7
5	From Neurons to Layers	7
6	Network Structure and Architecture	8
6.1	Matrix dimensions	8
7	How Many Parameters Does a Network Have?	9
8	Depth Versus Width	9
9	Feed-Forward Propagation	10
10	Worked Forward-Pass Example	10
11	The Loss Function	11
12	What Training Means Mathematically	11
13	Gradient Descent	11
14	Backpropagation: The Chain Rule Applied Efficiently	12
15	Why the Activation Derivative Matters	12
16	A Scalar Backpropagation Example	12
17	Backpropagation in Matrix Form	13

18 Full Backpropagation of the Numerical Example	14
18.1 Output-layer weight gradient	14
18.2 Propagating the gradient to the hidden layer	14
18.3 First-layer weight gradient	15
19 Parameter Updates in the Worked Example	15
20 The Complete Training Cycle	16
21 Mini-Batches	16
22 Epochs, Batches, and Updates	17
23 Vanishing Gradients	17
24 ReLU and Gradient Propagation	17
25 Weight Initialization	18
26 Choosing a Network Architecture	18
27 Output-Layer Design	19
27.1 Regression	19
27.2 Binary classification	19
27.3 Multiclass classification	19
28 One-Hot Targets and PyTorch Cross Entropy	20
29 Backpropagation Through a Deep Network	20
30 Why Backpropagation Is Efficient	21
31 Computational-Graph View	21
32 Gradient Descent, SGD, and Mini-Batch SGD	21
33 Momentum and Adam	22
34 The Learning Rate	22
35 Weights, Activations, Gradients, and Hyperparameters	23

36 A Useful Mental Model	23
37 Compact Mathematical Summary	23
38 Forward and Backward Passes as Mirror Images	24
39 Final Takeaways	24

1 The Core Idea: One Neuron

A neural network is easiest to understand as a **parameterized mathematical function** built by repeatedly applying two operations:

$$\text{linear transformation} \longrightarrow \text{nonlinear activation}$$

For a single neuron,

$$z = \mathbf{w}^\top \mathbf{x} + b, \tag{1}$$

followed by

$$a = \phi(z), \tag{2}$$

where

$$\begin{aligned} \mathbf{x} &= \text{inputs}, & \mathbf{w} &= \text{weights}, & b &= \text{bias}, \\ z &= \text{pre-activation}, & \phi &= \text{activation function}, & a &= \text{neuron output}. \end{aligned}$$

The weights determine how strongly each input matters. The bias shifts the neuron's threshold or operating region. The activation function determines how the neuron transforms the weighted input.

1.1 Numerical example

Let

$$\mathbf{x} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 0.5 \\ 0.7 \\ -0.2 \end{bmatrix}, \quad b = 0.3.$$

Then

$$\begin{aligned} z &= 0.5(2) + 0.7(-1) - 0.2(3) + 0.3 \\ &= 1 - 0.7 - 0.6 + 0.3 = 0. \end{aligned}$$

If the activation is ReLU,

$$\phi(z) = \max(0, z),$$

then

$$a = 0.$$

If instead the same neuron produced $z = 2$, then $a = \text{ReLU}(2) = 2$.

2 Why Activation Functions Are Necessary

Suppose all activations were removed. Consider two layers:

$$\mathbf{h} = W_1 \mathbf{x} + b_1$$

and

$$\mathbf{y} = W_2 \mathbf{h} + b_2.$$

Substituting the first into the second gives

$$\begin{aligned}\mathbf{y} &= W_2(W_1\mathbf{x} + b_1) + b_2 \\ &= W_2W_1\mathbf{x} + W_2b_1 + b_2.\end{aligned}$$

Define

$$W^* = W_2W_1, \quad b^* = W_2b_1 + b_2.$$

Then

$$\mathbf{y} = W^*\mathbf{x} + b^*$$

which is still one linear transformation. Therefore even a very deep stack of purely linear layers is mathematically equivalent to one linear layer. Nonlinear activations are what allow a neural network to approximate nonlinear functions and complex decision boundaries.

3 Common Activation Functions

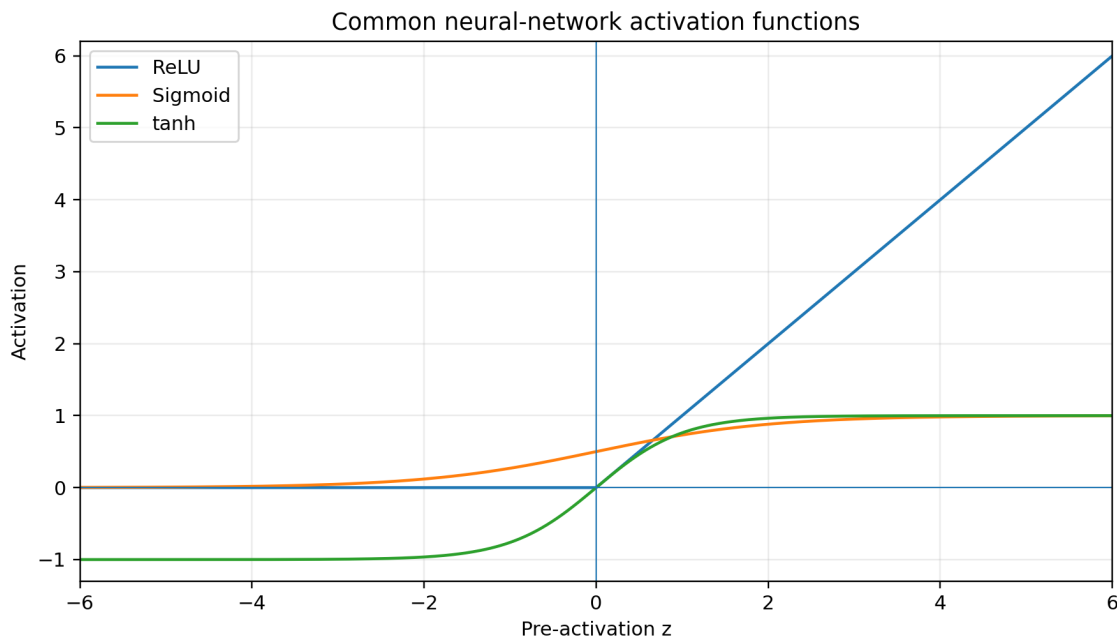


Figure 1: ReLU, sigmoid, and tanh over a common input range.

3.1 ReLU

The Rectified Linear Unit is

$$\text{ReLU}(z) = \max(0, z)$$

so

$$\text{ReLU}(z) = \begin{cases} z, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

Its derivative is

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0. \end{cases}$$

At exactly zero, software libraries choose a conventional derivative, commonly 0.

A useful property is that for an active neuron, such as $z = 5$, the derivative is 1. Therefore the activation itself does not automatically shrink the gradient.

3.2 Sigmoid

The sigmoid function is

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

with output satisfying $0 < \sigma(z) < 1$. Its derivative is

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

and its maximum possible derivative is 0.25. For example,

$$\sigma(0) = 0.5, \quad \sigma'(0) = 0.25.$$

But for large positive or negative z , the function saturates. For example,

$$\sigma(8) \approx 0.9997,$$

so $\sigma'(8)$ is close to zero. This is one source of the vanishing-gradient problem in deep sigmoid networks.

Sigmoid is therefore not usually the first choice for hidden layers, but it is natural at the output of a binary classifier.

3.3 Tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad -1 < \tanh(z) < 1.$$

Its derivative is

$$\tanh'(z) = 1 - \tanh^2(z).$$

Tanh is centered around zero, since $\tanh(0) = 0$, but it still saturates for sufficiently large positive or negative inputs.

3.4 GELU

GELU is common in Transformer architectures:

$$\text{GELU}(z) = z\Phi(z),$$

where Φ is the standard Gaussian cumulative distribution function. A common approximation is

$$\text{GELU}(z) \approx \frac{z}{2} \left[1 + \tanh \left(\sqrt{\frac{2}{\pi}} \left(z + 0.044715z^3 \right) \right) \right].$$

Unlike ReLU, GELU smoothly suppresses negative inputs rather than setting them abruptly to zero.

4 Why Activations Usually Follow Every Hidden Layer

Without activations, a chain such as

$$\mathbf{x} \rightarrow W_1 \rightarrow W_2 \rightarrow W_3$$

collapses to a single linear map $W_3W_2W_1\mathbf{x}$. With ReLU hidden activations,

$$\mathbf{h}_1 = \text{ReLU}(W_1\mathbf{x} + b_1),$$

$$\mathbf{h}_2 = \text{ReLU}(W_2\mathbf{h}_1 + b_2),$$

$$\mathbf{y} = W_3\mathbf{h}_2 + b_3,$$

the overall mapping is nonlinear and cannot generally be collapsed into one matrix multiplication.

A practical rule is:

Apply a nonlinear activation after each hidden layer. Do not automatically apply the same activation after the output layer; the output activation must match the prediction task.

Table 1: Typical output-layer choices.

Problem	Output dimension	Output activation
Regression	1 or more	Usually none
Binary classification	1	Sigmoid
Multiclass classification	K	Softmax
Multilabel classification	K	Independent sigmoids

5 From Neurons to Layers

Suppose a layer contains m neurons and receives n inputs. Collect the weights into

$$W \in \mathbb{R}^{m \times n}$$

and the biases into

$$\mathbf{b} \in \mathbb{R}^m.$$

The entire layer computes

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

followed by

$$\mathbf{a} = \phi(\mathbf{z})$$

where ϕ is applied elementwise.

For example, if

$$\mathbf{z} = \begin{bmatrix} -2 \\ 0.5 \\ 3 \end{bmatrix},$$

then ReLU gives

$$\mathbf{a} = \begin{bmatrix} 0 \\ 0.5 \\ 3 \end{bmatrix}.$$

6 Network Structure and Architecture

Consider the dense architecture

$$784 \rightarrow 128 \rightarrow 64 \rightarrow 10.$$

This means 784 input variables, 128 neurons in the first hidden layer, 64 neurons in the second hidden layer, and 10 output units.

For MNIST, each 28×28 image contains

$$28 \cdot 28 = 784$$

pixels, so a fully connected network commonly flattens the image into a vector of dimension 784 before feeding it to the first dense layer.

6.1 Matrix dimensions

If

$$\mathbf{x} \in \mathbb{R}^{784},$$

then for the first layer

$$W^{[1]} \in \mathbb{R}^{128 \times 784}, \quad b^{[1]} \in \mathbb{R}^{128},$$

so

$$z^{[1]} = W^{[1]}x + b^{[1]} \in \mathbb{R}^{128}.$$

For the second layer,

$$W^{[2]} \in \mathbb{R}^{64 \times 128},$$

therefore $a^{[2]} \in \mathbb{R}^{64}$. Finally,

$$W^{[3]} \in \mathbb{R}^{10 \times 64}$$

produces a vector of 10 logits or outputs.

7 How Many Parameters Does a Network Have?

For a dense layer with n_{in} inputs and n_{out} neurons,

$$N_{\text{weights}} = n_{\text{in}}n_{\text{out}}, \quad N_{\text{biases}} = n_{\text{out}}.$$

Thus

$$N_{\text{parameters}} = n_{\text{in}}n_{\text{out}} + n_{\text{out}}$$

For the architecture $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$:

$$784(128) + 128 = 100,480,$$

$$128(64) + 64 = 8,256,$$

$$64(10) + 10 = 650.$$

The total number of trainable parameters is

$$100,480 + 8,256 + 650 = 109,386.$$

Neurons themselves are not parameters. The learned parameters are primarily the weights and biases connecting them.

8 Depth Versus Width

Width is the number of neurons within a layer. For example,

$$784 \rightarrow 1024 \rightarrow 10$$

has one very wide hidden layer.

Depth is the number of successive transformations. For example,

$$784 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 10$$

is much deeper.

Depth allows the model to build hierarchical representations. In an image network, one may loosely think of

$$\text{pixels} \rightarrow \text{edges} \rightarrow \text{shapes} \rightarrow \text{higher-level representations}.$$

In tabular models the intermediate representations are less visually interpretable, but later layers still represent progressively more complicated nonlinear combinations of earlier features.

More depth or width does not automatically improve out-of-sample performance. Capacity must be balanced against optimization difficulty, overfitting, data volume, normalization, and regularization.

9 Feed-Forward Propagation

Suppose a network contains L layers. Define

$$a^{[0]} = x.$$

For layer l ,

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

and

$$a^{[l]} = \phi^{[l]}(z^{[l]})$$

The final prediction is

$$\hat{y} = a^{[L]}.$$

The forward pass is therefore the repeated evaluation

$$x = a^{[0]} \rightarrow z^{[1]} \rightarrow a^{[1]} \rightarrow z^{[2]} \rightarrow a^{[2]} \rightarrow \dots \rightarrow \hat{y}.$$

10 Worked Forward-Pass Example

Consider a tiny network

$$2 \rightarrow 2 \rightarrow 1.$$

Let

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad W^{[1]} = \begin{bmatrix} 0.5 & -0.2 \\ 0.3 & 0.8 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}.$$

Using ReLU,

$$z^{[1]} = W^{[1]}x + b^{[1]}.$$

The first hidden neuron receives

$$0.5(1) - 0.2(2) + 0.1 = 0.2,$$

and the second receives

$$0.3(1) + 0.8(2) - 0.1 = 1.8.$$

Therefore

$$z^{[1]} = \begin{bmatrix} 0.2 \\ 1.8 \end{bmatrix}, \quad a^{[1]} = \begin{bmatrix} 0.2 \\ 1.8 \end{bmatrix}.$$

Let the output layer be

$$W^{[2]} = \begin{bmatrix} 0.7 & -0.5 \end{bmatrix}, \quad b^{[2]} = 0.2.$$

Then

$$\begin{aligned} z^{[2]} &= 0.7(0.2) - 0.5(1.8) + 0.2 \\ &= 0.14 - 0.9 + 0.2 = -0.56. \end{aligned}$$

For binary classification, apply sigmoid:

$$\hat{y} = \sigma(-0.56) \approx 0.3635.$$

The model currently assigns about 36.35% probability to class 1.

11 The Loss Function

Training requires a scalar objective that quantifies prediction error.

For regression, a convenient form of squared error is

$$L = \frac{1}{2}(\hat{y} - y)^2,$$

for which

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y.$$

For binary classification, binary cross entropy is

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})].$$

In the worked example, if $y = 1$ and $\hat{y} = 0.3635$, then

$$L = -\log(0.3635) \approx 1.0118.$$

The objective of training is to modify the network parameters so that this loss decreases.

12 What Training Means Mathematically

Collect the parameters into

$$\theta = \{W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, \dots\}.$$

The network defines

$$\hat{y} = f(x; \theta),$$

so the loss is a function of the parameters:

$$L = L(\theta).$$

Training seeks an approximate solution to

$$\theta^* = \arg \min_{\theta} L(\theta).$$

Because a realistic network may contain millions or billions of parameters, optimization relies on gradients rather than exhaustive search.

13 Gradient Descent

For one scalar parameter w ,

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

where η is the learning rate.

For all parameters simultaneously,

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L.$$

The minus sign appears because the gradient points in the direction of greatest local increase in the loss. Gradient descent moves in the opposite direction.

14 Backpropagation: The Chain Rule Applied Efficiently

The main challenge is computing $\partial L / \partial w$ for every weight and bias in a deep network. Backpropagation solves this by repeatedly applying the chain rule from the output layer back toward the input.

For a scalar chain

$$w_1 \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \hat{y} \rightarrow L,$$

we have

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial a_1} \frac{\partial a_1}{\partial z_1} \frac{\partial z_1}{\partial w_1}$$

Each factor is a local derivative. Backpropagation organizes these local derivatives so intermediate results are reused rather than recomputed from scratch for every parameter.

15 Why the Activation Derivative Matters

If

$$a = \phi(z),$$

then the backward pass must cross the activation:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} = \frac{\partial L}{\partial a} \phi'(z).$$

Thus activation functions affect both the forward representation and the ability of gradients to propagate backward.

16 A Scalar Backpropagation Example

Suppose

$$z = wx + b, \quad a = \text{ReLU}(z), \quad L = \frac{1}{2}(a - y)^2.$$

Using the chain rule,

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w}.$$

Now

$$\frac{\partial L}{\partial a} = a - y, \quad \frac{\partial z}{\partial w} = x.$$

For $z > 0$, ReLU has derivative 1, so

$$\frac{\partial L}{\partial w} = (a - y)x.$$

For $z < 0$, the ReLU derivative is zero, hence

$$\frac{\partial L}{\partial w} = 0.$$

This explains why an inactive ReLU transmits no gradient for that observation.

17 Backpropagation in Matrix Form

Recall the forward equations

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}, \quad a^{[l]} = \phi(z^{[l]}).$$

Define the layer error signal

$$\delta^{[l]} = \frac{\partial L}{\partial z^{[l]}}.$$

Once $\delta^{[l]}$ is known, the weight and bias gradients are

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]}(a^{[l-1]})^\top$$

and

$$\frac{\partial L}{\partial b^{[l]}} = \delta^{[l]}.$$

The error propagates backward according to

$$\delta^{[l]} = \left((W^{[l+1]})^\top \delta^{[l+1]} \right) \odot \phi'(z^{[l]})$$

where $\odot$ denotes elementwise multiplication.

The term

$$(W^{[l+1]})^\top \delta^{[l+1]}$$

measures how much the neurons in layer l contributed to the error in the next layer. Multiplication by $\phi'(z^{[l]})$ then accounts for the sensitivity of each neuron's activation to its pre-activation.

18 Full Backpropagation of the Numerical Example

Return to the network

$$2 \rightarrow 2 \rightarrow 1$$

with

$$a^{[1]} = \begin{bmatrix} 0.2 \\ 1.8 \end{bmatrix}, \quad \hat{y} = 0.363547,$$

and target $y = 1$.

For sigmoid output combined with binary cross entropy,

$$\frac{\partial L}{\partial z^{[2]}} = \hat{y} - y.$$

Hence

$$\delta^{[2]} = 0.363547 - 1 = -0.636453.$$

18.1 Output-layer weight gradient

Using

$$W^{[2]} = \begin{bmatrix} 0.7 & -0.5 \end{bmatrix},$$

we obtain

$$\frac{\partial L}{\partial W^{[2]}} = \delta^{[2]} (a^{[1]})^\top = -0.636453 \begin{bmatrix} 0.2 & 1.8 \end{bmatrix}.$$

Therefore

$$\frac{\partial L}{\partial W^{[2]}} = \begin{bmatrix} -0.12729 & -1.14561 \end{bmatrix}$$

and

$$\frac{\partial L}{\partial b^{[2]}} = -0.636453.$$

The second connection receives a much larger gradient partly because the corresponding hidden activation was 1.8 rather than 0.2.

18.2 Propagating the gradient to the hidden layer

Because both hidden pre-activations are positive,

$$\text{ReLU}'(z^{[1]}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Thus

$$\begin{aligned}\delta^{[1]} &= (W^{[2]})^\top \delta^{[2]} \odot \text{ReLU}'(z^{[1]}) \\ &= \begin{bmatrix} 0.7 \\ -0.5 \end{bmatrix} (-0.636453) \odot \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} -0.445517 \\ 0.318226 \end{bmatrix}.\end{aligned}$$

The two hidden neurons receive gradients of opposite sign: the network has determined that moving their contributions in opposite directions would reduce the final loss.

18.3 First-layer weight gradient

Because

$$x^\top = \begin{bmatrix} 1 & 2 \end{bmatrix},$$

we have

$$\frac{\partial L}{\partial W^{[1]}} = \delta^{[1]} x^\top = \begin{bmatrix} -0.445517 \\ 0.318226 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix}.$$

Therefore

$$\frac{\partial L}{\partial W^{[1]}} = \begin{bmatrix} -0.445517 & -0.891034 \\ 0.318226 & 0.636453 \end{bmatrix}$$

and

$$\frac{\partial L}{\partial b^{[1]}} = \begin{bmatrix} -0.445517 \\ 0.318226 \end{bmatrix}.$$

At this point the gradient of the loss has been computed with respect to every parameter in the network.

19 Parameter Updates in the Worked Example

Let the learning rate be

$$\eta = 0.1.$$

The first-layer weights began at

$$W^{[1]} = \begin{bmatrix} 0.5 & -0.2 \\ 0.3 & 0.8 \end{bmatrix}.$$

Gradient descent gives

$$W_{\text{new}}^{[1]} = W_{\text{old}}^{[1]} - 0.1 \frac{\partial L}{\partial W^{[1]}},$$

so

$$W_{\text{new}}^{[1]} = \begin{bmatrix} 0.544552 & -0.110897 \\ 0.268177 & 0.736355 \end{bmatrix}$$

The first-layer biases become

$$b_{\text{new}}^{[1]} = \begin{bmatrix} 0.144552 \\ -0.131823 \end{bmatrix}.$$

For the output layer,

$$W_{\text{new}}^{[2]} = \begin{bmatrix} 0.7 & -0.5 \end{bmatrix} - 0.1 \begin{bmatrix} -0.127291 & -1.145615 \end{bmatrix},$$

hence

$$W_{\text{new}}^{[2]} = \begin{bmatrix} 0.712729 & -0.385439 \end{bmatrix},$$

and

$$b_{\text{new}}^{[2]} = 0.263645.$$

One complete training step has now finished.

20 The Complete Training Cycle

The training loop can be summarized as

$$\text{inputs} \xrightarrow{\text{forward}} \hat{y} \xrightarrow{\text{loss}} L \xrightarrow{\text{backprop}} \nabla_{\theta} L \xrightarrow{\text{optimizer}} \theta_{\text{new}}$$

Algorithmically:

1. Feed input X into the network.
2. Compute every $z^{[l]}$.
3. Compute every activation $a^{[l]}$.
4. Obtain the prediction $\hat{y}$.
5. Compute the loss $L(y, \hat{y})$.
6. Compute the output-layer gradient.
7. Propagate gradients backward through each layer.
8. Compute $\partial L / \partial W$ and $\partial L / \partial b$.
9. Update the parameters using the optimizer.
10. Repeat on subsequent batches.

21 Mini-Batches

In practice, many samples are processed simultaneously. If

$$X \in \mathbb{R}^{B \times d},$$

then B is the batch size. Rather than update after each observation, the optimizer often uses the mean batch loss

$$L_{\text{batch}} = \frac{1}{B} \sum_{i=1}^B L_i.$$

Its gradient is

$$\nabla_{\theta} L_{\text{batch}} = \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} L_i.$$

Then

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L_{\text{batch}}.$$

22 Epochs, Batches, and Updates

Suppose MNIST has 60,000 training images and the batch size is 100. Then

$$\frac{60,000}{100} = 600$$

batches are processed per epoch. One epoch means the model has passed through the full training set once, but there are approximately 600 parameter updates during that epoch.

After 20 epochs, the model has performed roughly

$$20 \times 600 = 12,000$$

updates. Therefore

one epoch $\neq$ one parameter update.

23 Vanishing Gradients

Backpropagation repeatedly multiplies derivatives. For sigmoid,

$$\sigma'(z) \leq 0.25.$$

If a gradient passes through ten layers with activation derivatives around 0.2, the derivative contribution alone is

$$0.2^{10} = 1.024 \times 10^{-7}.$$

A gradient that begins at order 1 near the output can therefore become extremely small by the time it reaches the first layers. Those early layers then learn very slowly. This is the classic vanishing-gradient problem.

24 ReLU and Gradient Propagation

For active ReLU units,

$$\text{ReLU}'(z) = 1,$$

so the activation itself does not attenuate the gradient. This is one reason ReLU helped make deeper networks easier to optimize.

However, if

$$z < 0,$$

then

$$\text{ReLU}'(z) = 0.$$

A neuron that remains on the negative side can stop receiving useful gradient updates. This is called a **dead ReLU**.

Leaky ReLU addresses this by using a small nonzero slope in the negative region:

$$\text{LeakyReLU}(z) = \begin{cases} z, & z > 0, \\ \alpha z, & z \leq 0, \end{cases}$$

where α may be, for example, 0.01.

25 Weight Initialization

If every weight in a layer were initialized to exactly zero, all neurons in that layer would initially compute the same output and receive identical gradients. They would continue to learn the same representation. Random initialization breaks this symmetry.

For ReLU networks, He initialization is common:

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right).$$

The goal is to keep the scale of activations and gradients reasonably stable as signals move through many layers.

For sigmoid or tanh networks, Xavier/Glorot initialization is common, with variance on the order of

$$\text{Var}(W) \approx \frac{1}{n_{\text{in}}}$$

or a related expression involving both fan-in and fan-out.

26 Choosing a Network Architecture

There is no universal formula for the correct number of layers or neurons. Architecture controls both capacity and optimization difficulty.

Increasing width increases the number of units available to represent patterns at a particular stage. Increasing depth increases the number of nonlinear transformations that can be composed.

However, a deeper network is not automatically better. It introduces more parameters, a harder optimization problem, potentially unstable gradients, greater overfitting capacity, and more sensitivity to learning rate, initialization, normalization, and regularization.

For a task such as MNIST, if

$$784 \rightarrow 128 \rightarrow 10$$

trains well but

$$784 \rightarrow 128 \rightarrow 64 \rightarrow 10$$

does not, the issue is usually not that two hidden layers are incapable of solving the task. The second network is more expressive. Failure more often points to an optimization or implementation issue, such as a missing activation, an inappropriate learning rate, an incorrect loss/output combination, bad initialization, tensor-shape mistakes, or inadvertently detached tensors.

27 Output-Layer Design

27.1 Regression

For an unrestricted scalar target,

$$\hat{y} = W^{[L]}a^{[L-1]} + b^{[L]}$$

is often used directly, with no output activation. For a financial return target, this is important because a ReLU output would incorrectly force

$$\hat{y} \geq 0.$$

27.2 Binary classification

Use one logit

$$z = Wa + b$$

and sigmoid

$$\hat{y} = \sigma(z),$$

so $0 < \hat{y} < 1$.

27.3 Multiclass classification

For K classes, the network produces logits $z_1, \dots, z_K$ and softmax converts them into probabilities:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

These satisfy

$$\sum_{k=1}^K p_k = 1.$$

For example, for ten classes a model might output

$$[0.01, 0.02, 0.03, 0.01, 0.80, 0.04, 0.02, 0.03, 0.02, 0.02].$$

The predicted class is the index with the largest probability.

28 One-Hot Targets and PyTorch Cross Entropy

For an MNIST digit with label 3, a one-hot mathematical representation is

$$\mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

Conceptually, cross entropy compares the target class distribution against the predicted softmax distribution.

In PyTorch, however, `CrossEntropyLoss` normally expects the integer class index directly, such as $y = 3$, rather than the one-hot vector. It internally combines the relevant log-softmax and negative log-likelihood calculations. The mathematical representation and the library API therefore differ slightly.

29 Backpropagation Through a Deep Network

For a three-layer network,

$$\begin{aligned} a^{[0]} &= x, \\ z^{[1]} &= W^{[1]}a^{[0]} + b^{[1]}, & a^{[1]} &= \phi(z^{[1]}), \\ z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]}, & a^{[2]} &= \phi(z^{[2]}), \\ z^{[3]} &= W^{[3]}a^{[2]} + b^{[3]}, & \hat{y} &= \psi(z^{[3]}). \end{aligned}$$

The forward pass moves from layer 1 to layer 3. The backward pass starts at the output:

$$\delta^{[3]} = \frac{\partial L}{\partial z^{[3]}},$$

then

$$\delta^{[2]} = (W^{[3]})^\top \delta^{[3]} \odot \phi'(z^{[2]}),$$

and

$$\delta^{[1]} = (W^{[2]})^\top \delta^{[2]} \odot \phi'(z^{[1]}).$$

At each layer,

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]}(a^{[l-1]})^\top.$$

Backpropagation is therefore recursive: once the gradient at layer $l + 1$ is known, the gradient at layer l can be obtained efficiently.

30 Why Backpropagation Is Efficient

Suppose a neural network has 10^7 parameters. A naive finite-difference gradient would perturb each parameter separately,

$$w_i \rightarrow w_i + \epsilon,$$

and recompute the loss. This would require millions of additional forward evaluations for one gradient estimate.

Backpropagation reuses intermediate results. The forward pass stores quantities such as $z^{[l]}$ and $a^{[l]}$, while the backward pass recursively reuses error signals from later layers. This is an instance of **reverse-mode automatic differentiation**, which is especially efficient when a function has many parameters but produces one scalar loss.

In PyTorch, the call

```
loss.backward()
```

triggers this gradient computation automatically for parameters connected to the computational graph.

31 Computational-Graph View

Consider

$$L = (wx + b)^2.$$

Let

$$z = wx + b.$$

Then

$$L = z^2, \quad \frac{\partial L}{\partial z} = 2z, \quad \frac{\partial z}{\partial w} = x.$$

Hence

$$\frac{\partial L}{\partial w} = 2zx.$$

A framework decomposes a complicated model into many such elementary operations, computes the forward values, and traverses the graph backward to accumulate derivatives.

32 Gradient Descent, SGD, and Mini-Batch SGD

Full-batch gradient descent uses all N observations to compute

$$\nabla L = \frac{1}{N} \sum_{i=1}^N \nabla L_i$$

before each update.

Pure stochastic gradient descent uses one observation at a time:

$$\theta \leftarrow \theta - \eta \nabla L_i.$$

In deep learning, “SGD” often refers in practice to mini-batch SGD:

$$\nabla L_B = \frac{1}{B} \sum_{i \in B} \nabla L_i.$$

Typical batch sizes might be 32, 64, 128, or 256. Mini-batches offer a useful compromise between computational efficiency and gradient noise.

33 Momentum and Adam

Basic SGD uses

$$\theta_{t+1} = \theta_t - \eta g_t, \quad g_t = \nabla_{\theta} L_t.$$

Momentum forms a smoothed gradient direction,

$$v_t = \beta v_{t-1} + (1 - \beta) g_t,$$

then updates

$$\theta_{t+1} = \theta_t - \eta v_t.$$

Adam tracks both a first-moment and a second-moment estimate:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

After bias correction, the update is approximately

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Backpropagation and Adam are not alternatives. Backpropagation computes the gradients; Adam decides how to turn those gradients into parameter updates.

34 The Learning Rate

The learning rate controls the update magnitude:

$$\Delta \theta = -\eta \nabla L.$$

If η is too small, training can be unnecessarily slow. If η is too large, the optimizer can repeatedly overshoot low-loss regions or diverge. The learning rate is therefore among the most consequential hyperparameters in neural-network training.

35 Weights, Activations, Gradients, and Hyperparameters

Table 2: Key quantities and whether they are learned.

Quantity	Example	Meaning	Learned?
Input	x	Data entering the network	No
Weight	W	Connection strength	Yes
Bias	b	Additive offset	Yes
Pre-activation	$z = Wx + b$	Weighted signal before nonlinearity	No
Activation	$a = \phi(z)$	Neuron output	No
Prediction	$\hat{y}$	Final network output	No
Loss	L	Scalar prediction error	No
Gradient	$\partial L / \partial W$	Sensitivity of loss to a parameter	No
Learning rate	η	Update step-size hyperparameter	No

The network learns weights and biases. Activations are temporary values created during a forward pass, and gradients are temporary sensitivity values created during the backward pass.

36 A Useful Mental Model

The three main conceptual stages are:

Forward pass: given the current parameters, what does the network predict?

$$x \longrightarrow \hat{y}.$$

Loss: how wrong is that prediction?

$$(y, \hat{y}) \longrightarrow L.$$

Backward pass: how did each parameter contribute to the error, and in what direction should it move?

$$L \longrightarrow \nabla_{\theta} L.$$

The optimizer then maps

$$\theta \longrightarrow \theta_{\text{new}}.$$

37 Compact Mathematical Summary

A standard feed-forward neural network with L layers obeys

$$a^{[0]} = x$$

and

$$a^{[l]} = \phi^{[l]} \left(W^{[l]} a^{[l-1]} + b^{[l]} \right)$$

for $l = 1, \dots, L$. The prediction is $\hat{y} = a^{[L]}$, and training minimizes a loss $L(y, \hat{y})$.

Backpropagation starts from

$$\delta^{[L]} = \frac{\partial L}{\partial z^{[L]}}$$

and recursively computes

$$\delta^{[l]} = (W^{[l+1]})^\top \delta^{[l+1]} \odot \phi'(z^{[l]})$$

with

$$\nabla_{W^{[l]}} L = \delta^{[l]} (a^{[l-1]})^\top, \quad \nabla_{b^{[l]}} L = \delta^{[l]}.$$

The parameters are then updated, for basic gradient descent, as

$$\begin{aligned} W^{[l]} &\leftarrow W^{[l]} - \eta \nabla_{W^{[l]}} L, \\ b^{[l]} &\leftarrow b^{[l]} - \eta \nabla_{b^{[l]}} L. \end{aligned}$$

38 Forward and Backward Passes as Mirror Images

During the forward pass,

$$a^{[l-1]} \xrightarrow{W^{[l]}, b^{[l]}} z^{[l]} \xrightarrow{\phi} a^{[l]}.$$

During the backward pass,

$$\frac{\partial L}{\partial a^{[l]}} \xrightarrow{\phi'(z^{[l]})} \frac{\partial L}{\partial z^{[l]}} \xrightarrow{(W^{[l]})^\top} \frac{\partial L}{\partial a^{[l-1]}}.$$

Forward propagation uses W to send information to later units. Backpropagation naturally uses $W^\top$ to distribute responsibility for the final error back toward earlier units.

$$X \xrightarrow{\text{feed-forward}} \hat{Y} \rightarrow L \xrightarrow{\text{backprop}} \nabla_\theta L \xrightarrow{\text{optimizer}} \theta_{\text{new}}$$

39 Final Takeaways

A feed-forward neural network alternates affine transformations and nonlinear activations. Its architecture determines which representations it can construct, while the forward pass evaluates the current function represented by its parameters. The loss converts prediction quality into a scalar objective. Backpropagation efficiently applies the chain rule to compute the effect of every parameter on that loss. An optimizer then updates the weights and biases using those gradients.

The central equations are therefore:

$$\begin{aligned} z^{[l]} &= W^{[l]} a^{[l-1]} + b^{[l]}, \\ a^{[l]} &= \phi(z^{[l]}), \end{aligned}$$

$$\delta^{[l]} = (W^{[l+1]})^\top \delta^{[l+1]} \odot \phi'(z^{[l]}),$$

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^\top,$$

$$W^{[l]} \leftarrow W^{[l]} - \eta \frac{\partial L}{\partial W^{[l]}}.$$

Together, these equations capture the mathematical core of a standard dense neural network.